Gridiron Intelligence: Multi-Agent Football Scouting Application

Matthew Shaver Elliott Kervin

1 Abstract

College football programs face a persistent challenge in personnel evaluation: relevant information about recruits and transfer candidates is fragmented across databases, media outlets, and scouting services. Gridiron Intelligence addresses this problem by integrating a LangGraph-based multi-agent orchestration framework with a Gemini model cascade, a Supabase data lake, the College Football Data (CFBD) API, and Tavily web search to produce automated, evidence-grounded scouting reports. The system separates identity resolution, parallel data enrichment, and narrative synthesis into distinct pipeline stages, enabling fast, sub-cent report generation with explicit hallucination guardrails.

2 Introduction

College football programs make high-stakes personnel decisions (recruiting commitments, transfer acquisitions, and depth-chart management) based on information that is inherently scattered. A coaching staff evaluating a single prospect might consult a recruiting service, a statistics database, a regional media outlet, and internal notes, then manually synthesize those sources into a coherent assessment. At scale across an entire roster cycle, this process is both time-intensive and inconsistent.

The emergence of large language models with agentic tool-calling capabilities creates an opportunity to automate this workflow. By pairing structured database lookups with real-time web retrieval and grounded LLM synthesis, it is now feasible to generate a contextually rich scouting report in seconds rather than hours. The key engineering challenge is ensuring that speed does not come at the cost of accuracy: the system must respect the boundaries of its evidence rather than filling gaps with hallucination.

Gridiron Intelligence was built to explore this opportunity. It exposes three primary workflows (a recruiting portal, a transfer portal, and an open chat interface), each backed by a multi-agent orchestration graph that coordinates structured data access, web enrichment, and final synthesis through a cost-aware pair of Gemini models. The remainder of this report describes the architecture, data foundation, technical implementation, and evaluation results.

3 Methodology

Gridiron Intelligence is organized into four broad layers that keep the user interface separate from orchestration, data access, and agent reasoning. This separation supports both deterministic report generation and flexible multi-step analysis.

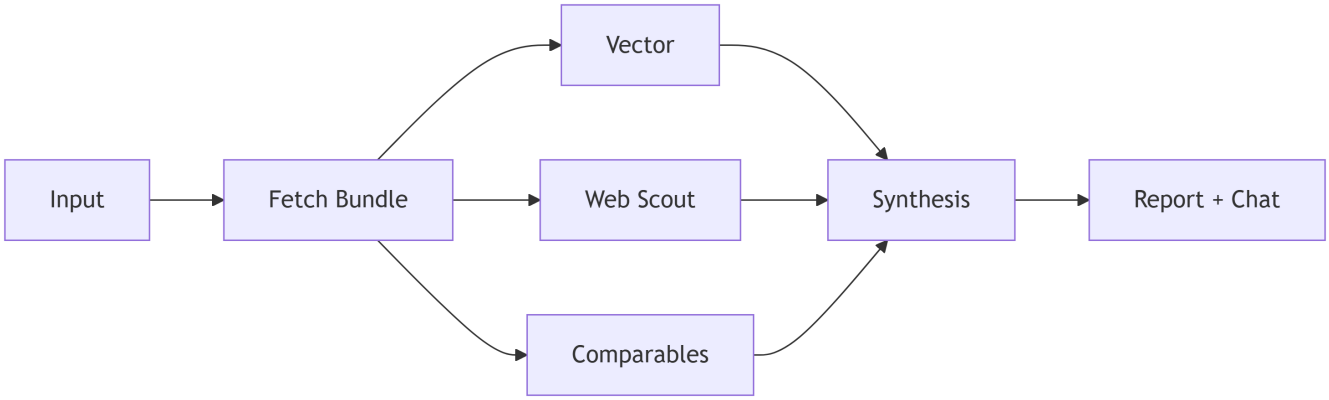
- **UI layer:** Handles page navigation, user input, and report display.
- **Orchestration layer:** Routes requests to the appropriate workflow (recruiting, transfer, or open chat) and manages the multi-agent execution graph.
- **Data and tool layer:** Pulls structured records from Supabase, live statistics from the CFBD API, and current context from Tavily web search.
- **Agent layer:** Coordinates planning, evidence gathering, and narrative synthesis.

3.1 Workflow Orchestration

The system uses two complementary orchestration patterns depending on the use case.

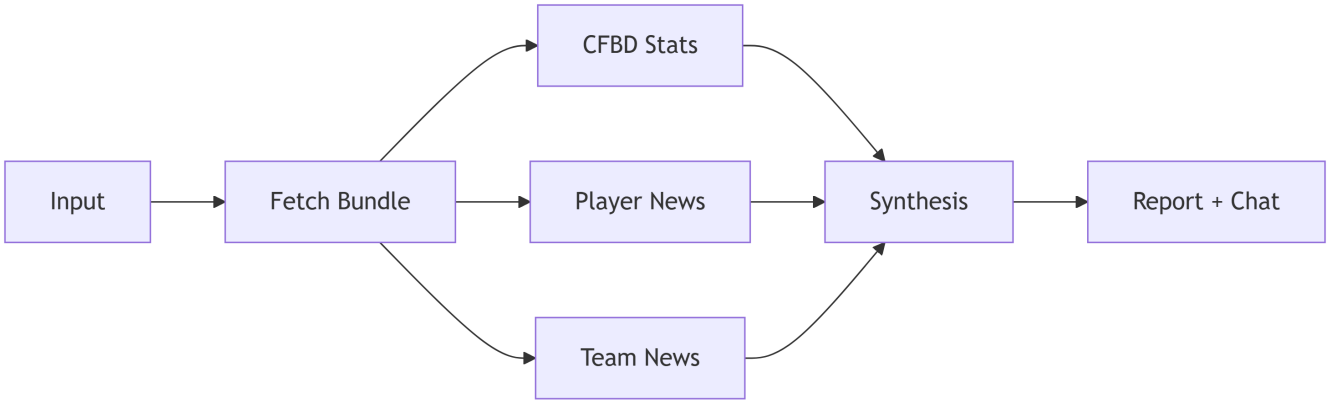
3.1.1 Recruiting Portal

The recruiting workflow fetches a structured player bundle from Supabase, then enriches it in parallel from vector insights, web context, and historical comparables before invoking the final synthesis model.



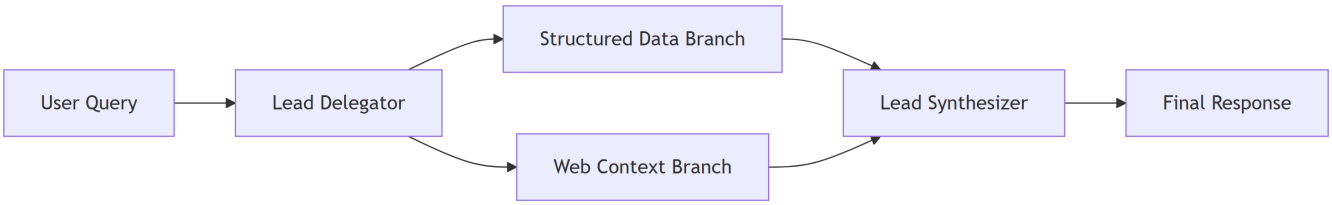
3.1.2 Transfer Portal

The transfer workflow resolves the player identity via Supabase, then gathers supporting evidence from three parallel branches before synthesizing the final report.



3.1.3 Open Chat

Open chat routes follow-up questions through a delegator that dispatches to specialized workers, then combines their outputs in the final synthesizer.



3.2 State and Memory Contract

The system maintains session stability through a dedicated state model. To prevent context growth from degrading response quality, it uses bounded memory via truncation rather than summarization.

- **State compaction:** Only the most recent turns, trace entries, errors, and citations are retained across turns.
- **Bulky context removal:** Large intermediate data objects (e.g., raw API JSON, web payloads) are cleared after they are no longer needed for the current turn.
- **Session scope:** Open chat is seeded from the current report context when available but does not carry memory across separate sessions.

4 Data Used

The effectiveness of scouting evaluations depends on precise player identification and the ability to merge static historical records with live performance metrics. The system prioritizes a SQL-first approach to ensure reliability and traceability.

4.1 Multi-Source Data Integration

Data is aggregated from four distinct channels:

- **Supabase (Primary Data Lake):** Stores canonical player bundles, historical physical data, and model prediction thresholds.
- **College Football Data (CFBD) API:** Provides live college-level statistics including seasonal totals and per-game usage metrics.
- **Tavily Web Search:** Supplies structured, real-time web context including full-text citations and relevance scores. The system uses Tavily for injury updates, transfer rumors, and team depth-chart news.
- **Vector Datastore:** Contains embedded qualitative "factoids" retrieved via similarity search to enrich prospect profiles.

4.2 Data Characteristics

The system is built on a master dataset covering prospects from 2015 to 2028. Modeling was performed on recruits from 2015 to 2022, tracking college outcomes. Future recruits (2026–2028) are available for evaluation in the app. Transfer candidates are limited to players who appeared in the 2025 season. For performance and relevance, the application defaults to a "Top 100" window by rating for current recruiting classes and requires users to search for a transfer candidate by name and position, avoiding long render times that would result from pre-loading full player lists.

Dataset / Table	Scope	Primary Use
Recruit Master	2026–2028	Prospect lookup for recruiting reports
College Master	2025	Player lookup for transfer reports
CFBD Statistics	Live API	Real-time usage and performance bars
Model Scores	Projected	Probability-based success forecasting
Factoid Vectors	Qualitative	Semantic enrichment of scouting reports

5 Concepts Implemented

This section maps each required concept to its concrete implementation: Prompting, RAG, Tooling, Multi-modality, Agent Design, Safety, Security, APIs, and Scaling.

5.1 Prompt Engineering and Grounding

The application uses a two-stage prompting architecture:

- **Upstream summarization prompts:** Heavily constrained prompts that force the model to return plain markdown bullet points when parsing raw web HTML or verbose API JSON. Temperature is set to 0.0 for deterministic output, and the system prompt explicitly forbids the model from including any information not present in the input data.
- **Downstream synthesis prompts:** The final synthesis prompt combines upstream summaries with strict grounding instructions, a temporal wrapper that injects the current date for recency-aware reasoning, and an explicit instruction to omit missing data rather than guess.

5.2 RAG and Vector Retrieval

The vector factoid store is queried via semantic similarity search against a Supabase pgvector table. When a player record is loaded, position and state filtered factoids are retrieved and injected into the synthesis prompt as grounding context. This retrieval-augmented generation (RAG) layer provides qualitative historical insight that complements the structured statistical data without requiring the full dataset to be embedded in the prompt.

5.3 Agentic Tool Calling

The system relies on structured tool wrappers to interface with the CFBD API and Tavily search endpoints.

- **Schema enforcement:** Tools are defined with strict input schemas, ensuring the LLM passes correctly formatted arguments.
- **Fail-safe execution:** If an external API times out or a web search returns sparse data, the tool returns a controlled fallback string (e.g., `"No context available at this time"`) rather than raising an unhandled exception, allowing the orchestration graph to complete the report gracefully.

5.4 Multi-modality

In addition to generating narrative text, the application renders visual outputs alongside each scouting report. In the recruiting portal the players' college projections, via a custom model, are diplayed in a card and passed to the model. The transfer report includes an interactive usage line chart and stat bar chart, as well as detailed stat tables. These visual components are generated dynamically from the same structured player bundles that feed the LLM synthesis, giving users both a narrative and a data-visual view of each candidate.

5.5 Agent Design

The orchestration layer is implemented as a LangGraph-based directed acyclic graph. Each node corresponds to a discrete agent responsibility: identity resolution, parallel data gathering, summarization, or final synthesis. The recruiting and transfer graphs use parallel branches to reduce generation time while still

collecting multi-source context.

5.6 Safety and Security

Agentic systems are uniquely vulnerable to payload poisoning from scraped web data and routing errors generated by the model itself.

- **Routing validation:** When the Open Chat feature is used, the delegator generates a routing plan that is validated against an expected schema before execution proceeds. Malformed routing instructions halt execution rather than propagating downstream.
- **Payload sanitization:** Raw web content passes through a sanitization block that strips unsafe HTML, hidden scripts, and anomalous code blocks before entering the agent state. This protects the synthesis prompt from prompt-injection attacks embedded in sports news articles.
- **Harmful Prompting:** The structured system prompt, along with safeguards built into the Gemini models, was demonstrated to prevent harmful or toxic outputs (i.e. leaking keys or system prompt), even when the user attempted to elicit such responses through Open Chat.

5.7 APIs and External Integrations

The system integrates three external APIs: the **CFBD API** for structured football statistics, **Tavily** for real-time web search with citation support, and the **Gemini API** via Google AI for both the summarization and synthesis models. Supabase provides the managed PostgreSQL database and pgvector extension backing the primary data lake and vector store.

5.8 Scaling Strategies

To balance cost and quality across the pipeline, the system uses a cost-aware model cascade:

- **Upstream tasks** (parsing web HTML, filtering CFBD JSON, extracting structured bullets) are routed to **Gemini 3.1 Flash-Lite**, a fast, low-cost model optimized for high-throughput parsing.
- **Final synthesis** (the full narrative scouting report) is reserved for **Gemini 3 Flash**, which provides deeper reasoning and higher output quality.

This split, combined with prompt-enforced payload caps (24,000 characters upstream) and bounded state compaction, keeps each report economical to generate without sacrificing output quality.

6 Experiments

To assess the performance of Gridiron Intelligence, we conducted experiments focused on orchestration topology and model cascade strategy.

6.1 Sequential vs. Parallel Orchestration

An initial implementation processed all data-gathering branches sequentially: CFBD stats, player news search, and team news search. This produced correct results but introduced noticeable latency as each branch waited on the previous one. Refactoring the external data-gathering branches to run concurrently using `ThreadPoolExecutor` within a LangGraph fan-out node reduced total report generation time substantially without changing output quality.

6.2 Tavily vs DuckDuckGo Search

Initially we utilized a DuckDuckGo (DDG) tool for web search as this allowed unlimited queries without cost. This was productive for the initial set up and early development. However, a later pivot was made to the more robust Tavily engine, which is designed for use with LLMs. The DDG tool often returned sparse results with limited relevance to the query, while Tavily provided richer payloads, more up to date information and higher relevance. The improved web context from Tavily led to more accurate and detailed scouting reports.

6.3 Summarization Quality Iterations

We iteratively refined the upstream web queries and summarization prompts, increasing the output token budget and expanding the entity-extraction guidelines to capture more specific facts from the web. Each iteration was validated manually against known-truth context for Charlotte 49ers roster and coaching activity.

7 Results

We evaluated the system across three dimensions: latency, cost efficiency, and context relevance. This was done via diagnostic outputs for each generated report which allowed for in depth analysis of the intermediate outputs from each stage in the process and cost/latency tracking for each API call.

7.1 Latency

Under typical conditions (database retrieval, parallel web scraping, summarization, and final synthesis), a complete scouting report is generated in **20 to 30 seconds**. The implementation of Tavily increased latency, due to richer retreived context, but this tradeoff was deemed acceptable given the substantial improvement in report quality. The parallel orchestration of data-gathering branches was critical to keeping latency within this range.

7.2 Cost Efficiency

On average, generating a complete multi-agent scouting report costs **less than $0.01 USD** in API token usage. This sub-cent cost reflects the effectiveness of the model cascade. Routing high-volume parsing to Gemini 3.1 Flash-Lite and capping upstream prompt payloads at 24,000 characters prevents the token explosion that would occur if all stages used the full synthesis model without payload constraints.

7.3 Context Relevance

Manual review of raw Tavily payloads against generated summaries showed consistent extraction of relevant entities (player names, school names, coaching staff, injury reports) after the summarization prompt refinements. Charlotte 49ers context (coaching changes, recent roster shifts) served as a repeatable ground-truth benchmark across evaluation cycles.

7.4 Agentic Reasoning

Open Chat multi-turn stress tests confirmed that the delegator correctly routes follow-up questions to the appropriate data branch in the majority of cases.

8 Conclusion

Gridiron Intelligence demonstrates that RAG, agentic tool calling, and multi-agent orchestration can be applied to a focused, production-grade football scouting problem. By separating identity management in the database layer, enforcing fan-out/fan-in graph topologies, and using a cost-aware model cascade, the system produces rich, evidence-grounded scouting reports quickly and economically. The architecture confirms that multi-agent reasoning can be both fast and affordable when properly constrained by bounded memory, payload caps, and strict prompt grounding.

9 Limitations and Future Work

While the fan-out/fan-in architecture stabilizes the reporting process, extensive human validation revealed several inherent limitations and opportunities for future improvement.

- **Web result quality dependency:** The system's accuracy is bounded by the quality of Tavily search results. Out-of-date or irrelevant articles are passed directly to the synthesis model, occasionally skewing the narrative. Tavily's recency filters proved unreliable, as most results lack clear publication dates.
- **Stubborn hallucinations:** Despite strict prompting constraints, the model occasionally generates incorrect claims; for example, assuming a player has entered the transfer portal when they have not. When challenged on these errors in Open Chat, the model sometimes doubles down and remains confidently incorrect despite repeated correction.
- **Future: internal enterprise deployment:** The ideal future state is deployment within a closed team environment, replacing generic web searches with trusted internal scouting notes, proprietary practice data, and vetted media sources. This would eliminate the public web dependency and substantially improve factual reliability.

10 References

- Google. (2026). Gemini API documentation. Google AI for Developers. https://ai.google.dev/gemini-api/docs
- LangChain, Inc. (2026). LangChain: Build reliable AI applications. https://www.langchain.com/
- Streamlit Inc. (2026). Streamlit documentation. https://docs.streamlit.io/
- College Football Data. (2026). CFBD API reference. https://api.collegefootballdata.com/
- Tavily. (2026). Tavily web search API documentation. https://docs.tavily.com/welcome
- Supabase. (2026). Supabase documentation. https://supabase.com/docs